

VoltGrid — Platform Technical Stack

Complete Technical Architecture, Frontend, Backend, Database, Security, APIs, and AI/Algorithms

Platform Name	Core Framework	Runtime Environment	Primary Region
VoltGrid v0.1.0	Next.js 16 + React 19	Node.js v24 (IPv4)	Tamil Nadu & S. India

1. System Overview & High-Level Architecture

VoltGrid is an intelligent electric vehicle (EV) energy and route optimization platform engineered for multi-class electric vehicles (2-wheelers and 4-wheelers) across Tamil Nadu and Southern India corridors. Unlike generic map utilities that calculate pure road distances, VoltGrid treats battery state-of-charge (SoC), vehicle payload dynamics, ambient weather, charger operational uptime, and real-time charging-point availability at estimated time of arrival (ETA) as first-class decision constraints.

The platform provides multi-tenant portal experiences for **Drivers**, **Charge Point Operators (CPOs)**, and **Community Charging Hosts**, backed by an interactive presentation sandbox for live outage simulation.



2. Frontend Architecture & Technology Stack

VoltGrid's frontend is constructed with the cutting-edge React 19 and Next.js 16 App Router foundation, optimized for sub-second page transitions, dynamic geospatial rendering, and responsive dark-mode visualization.

LAYER / CONCERN	TECHNOLOGY / PACKAGE	VERSION	PURPOSE & ARCHITECTURAL DETAILS
Core Framework	<code>next</code> (App Router)	16.3.2	React Server Components (RSC), route handlers, dynamic routes, and fast edge optimizations.
View Engine	<code>react</code> , <code>react-dom</code>	19.2.8	Latest concurrent React 19 features, client actions, and dynamic hydration.
Language	<code>typescript</code>	^5.0.0	Strict compile-time static type safety across schemas, models, and UI props.
Styling Engine	<code>tailwindcss</code> , <code>@tailwindcss/postcss</code>	^4.0.0	Tailwind CSS v4 with PostCSS, custom volt-cyan/emerald palette, dark mode, and glassmorphism.
CSS Utilities	<code>clsx</code> , <code>tailwind-merge</code>	2.1.1 / 3.3.1	Deterministic utility class merging via <code>cn()</code> utility helper.
Variant Management	<code>class-variance-authority</code>	^0.7.1	Type-safe declarative component variants (buttons, badges, modal cards).
Iconography	<code>lucide-react</code>	^0.544.0	Accessible, tree-shakeable SVG icons for vehicles, plugs, telemetry, and alerts.
Geospatial Maps	<code>leaflet</code> , <code>react-leaflet</code> , <code>react-leaflet-cluster</code>	1.9.4 / 5.0.0 / 4.1.3	Interactive maps with dynamic SSR-safe rendering (<code>MapCanvas.tsx</code>), clustering for 925+ POIs.
Map Base Tiles	CartoDB Dark Matter / Positron	Cloud	High-contrast raster/vector tiles optimized for night-time EV navigation.
Data Visualizations	<code>recharts</code>	^3.2.1	Interactive charts for battery discharge curves, grid demand profiles, and occupancy trends.
AI Assistant	VoltBot AI Assistant	Custom Widget	Context-aware interactive bot (<code>VoltBotAssistant.tsx</code>) with tailored role-specific prompts.
Payment Modal	Razorpay <code>checkout.js</code>	Client SDK	Seamless client payment popup with fallback simulation for sandbox keys.

Application Portals & Page Routes (27 Pages)

- **Driver Portal:** `/driver/find` (Station locator), `/driver/book` & `/driver/bookings` (Slot reservation & QR passes), `/driver/route` (Live in-transit HUD), `/driver/session` (Live charging telemetry & billing), `/driver/profile` (Vehicle specifications & presets).
- **Operator Portal:** `/operator/dashboard` (Grid metrics & station uptime), `/operator/network` (Hardware & transformer management), `/operator/bookings` (Slot revenue analytics), `/operator/sos` (Emergency dispatch & stranded driver escalations).
- **Host Portal:** `/host/register` (Residential/commercial host onboarding), `/host/earnings` (Tariff setting & payout ledger), `/host/community` (P2P host network guidelines).
- **Mobility & Core Views:** `/` & `/planner` (EV multi-stop route planner), `/route` (Route breakdown & stop cards), `/stations` & `/stations/[id]` (Station analytics & ETA predictions), `/demo` (Evaluator simulation sandbox), `/helpline` (Emergency tow network), `/login` & `/signup` (Auth portal).

3. Backend Architecture & Server Runtime

VoltGrid's backend operates as a decoupled service layer embedded inside Next.js 16 App Router Route Handlers, executed via a hardened Node.js HTTP server.

Custom Node.js Server Wrapper (`server.mjs`)

Constructed around `node:http` to resolve Next.js 16 keep-alive connection resets (`ERR_EMPTY_RESPONSE`). Explicitly closes upgrade sockets with valid HTTP status lines (426 / 403), sets `keepAliveTimeout = 0` , and binds concurrently to target port (`43123`) and fallback port (`3000`).

Authentication & RBAC (`src/lib/auth/`)

Dual-layer session tracking using browser `localStorage` synchronized with HTTP cookies (`voltgrid_role` , `voltgrid_session`). Protected portal layouts enforce role boundaries (`driver` , `operator` , `host`) via `PortalGuard` , with automatic onboarding redirects.

Complete Inventory of Backend REST APIs (16 Endpoints)

VERB	ENDPOINT PATH	ZOD SCHEMA VALIDATION	DESCRIPTION & FUNCTIONAL PIPELINE
POST	<code>/api/optimize-route</code>	<code>tripRequestSchema</code>	Calculates first-principles energy-aware multi-stop EV route, stop selection, and arrival SoC.
POST	<code>/api/reroute</code>	<code>rerouteSchema</code>	Autonomously recalculates routes in-transit when a reserved charger suffers an unexpected outage.
POST	<code>/api/predict-availability</code>	<code>predictSchema</code>	Predicts charger availability probability at driver ETA using logistic regression.
GET	<code>/api/route</code>	<code>optimizeQuerySchema</code>	Fetches point-to-point road geometry and duration from OSRM driving engine.
GET	<code>/api/stations</code>	Strict empty query	Returns list of all active charging stations within operational bounding boxes.
GET	<code>/api/stations/[id]</code>	<code>stationIdPathSchema</code>	Returns live station status, power capacity, queue duration, and connector specs.
GET	<code>/api/stations/coverage</code>	Strict empty query	Aggregates the combined network of 925+ verified charging stations across TN and Karnataka.
GET	<code>/api/stations/tamilnadu</code>	Strict empty query	Returns statewide corridor charging hubs across Tamil Nadu highways.
GET	<code>/api/vehicles</code>	Strict empty query	Returns catalogue of 26 calibrated Indian 2W and 4W electric vehicles with battery parameters.
GET	<code>/api/places</code>	<code>placesQuerySchema</code>	Geocoding & autocomplete with local seeded landmark cache and Nominatim fallback.
GET	<code>/api/grid/windows</code>	Strict empty query	Computes renewable grid energy charging windows and off-peak tariff periods.
GET	<code>/api/dashboard/metrics</code>	Strict empty query	Aggregates real-time network operational KPIs for the operator console.
POST	<code>/api/simulate/scenario</code>	<code>scenarioSchema</code>	Injects sandbox scenarios (<code>fail-recommended</code> , <code>high-demand</code> , <code>traffic</code> , <code>reset</code> , <code>demo-clock</code>).
POST	<code>/api/simulate/station-status</code>	<code>simulateStatusSchema</code>	Manually overrides individual station availability status (available, busy, offline).
POST	<code>/api/payments/razorpay/create-order</code>	JSON Body Validation	Creates Razorpay order with auto-fallback to simulated mock order for test development keys.
POST	<code>/api/payments/razorpay/verify-payment</code>	JSON Body Validation	Cryptographic HMAC-SHA256 signature verification matching Razorpay security guidelines.

4. Database & Storage Architecture

VoltGrid utilizes a hybrid storage architecture: an ultra-fast in-memory simulation store for hackathons/evaluations, persistent disk caching for statewide POIs, client-side localStorage/cookies for sessions, and a comprehensive PostgreSQL + PostGIS production schema.

STORAGE TIER	UNDERLYING ENGINE	USAGE & STORED ENTITIES
Simulation Store	In-Memory TS Singleton	Live queue lengths, temporary station outages, traffic multipliers, and synthetic clock state.
Statewide POI Cache	Local JSON Disk Cache	<code>.data/station-coverage.json</code> storing 883 Open Charge Map stations with 24-hr TTL refresh.
Client Storage	<code>localStorage</code> & Cookies	Driver/Operator/Host user sessions (<code>voltgrid_user_session_v1</code>) and booking ledgers (<code>voltgrid_bookings_v2</code>).
Production Database	PostgreSQL + PostGIS (SQL DDL)	Full relational model in <code>db/schema.sql</code> with spatial GIST indexing on geographic coordinates.

Production Database Tables & Spatial Indexing (`db/schema.sql`)

- `vehicles` : Model specs (battery kWh, base Wh/km, connector type, safety reserve, charge efficiency, weight).
- `charging_stations` : Spatial point entity (`GEOGRAPHY(POINT, 4326)`) indexed with `GIST (location)` .
- `chargers` : Individual EVSEs linked via foreign key to `charging_stations(id)` with OCPI identifiers.
- `station_status` & `availability_history` : Dynamic telemetry with compound index (`station_id, captured_at`) .
- `trip_requests` & `routes` : Historical journey queries with corridor geometry `GEOGRAPHY(LINESTRING, 4326)` .
- `route_stops` & `predictions` : Intermediate charging waypoints and ML inference log with JSONB factors.

5. Security & Defensive Engineering

VoltGrid adheres to strict defense-in-depth principles across input handling, network transport, abuse protection, and payment validation.

Input Validation & DoS Protection

- **Zod Runtime Parsing:** Strict schema validation rejects payloads with unexpected keys using `.strict()`.
- **128 KiB Body Cap:** Incoming JSON streams are strictly capped at 128 KiB to eliminate memory exhaustion vectors.
- **Identifier Sanitization:** Station IDs are strictly regex-validated (`/^[A-Za-z0-9_-]+$`).

Abuse Prevention & CORS Control

- **In-Memory Rate Limiter:** Token-bucket limiter enforcing 120 req/min general limit, with stricter reroute/simulation ceilings.
- **Proxy Guard:** Forwarded IPs trusted only when `TRUST_PROXY=1` is explicitly enabled.
- **Deny-by-Default CORS:** Wildcard `*` prohibited. Strictly allows configured `NEXT_PUBLIC_APP_URL` and localhost.

HTTP Security Headers & Content Security Policy (`next.config.ts`)

- `X-Content-Type-Options: nosniff` — Prevents MIME-type sniffing.
- `X-Frame-Options: DENY` — Prevents clickjacking attacks in iframes.
- `Strict-Transport-Security: max-age=63072000; includeSubDomains; preload` — Two-year HSTS enforcement.
- `Referrer-Policy: strict-origin-when-cross-origin` — Limits referrer information leakage.
- `Permissions-Policy: camera=(), microphone=(), geolocation=(self)` — Restricts sensitive hardware APIs.
- `Content-Security-Policy (CSP)` : Whitelists connect-src to authorized hosts (OSRM, Open Charge Map, Nominatim, Overpass, Razorpay).

6. External APIs & Third-Party Integrations

SERVICE	PROTOCOL	HOST / ENDPOINT	AUTHENTICATION	PURPOSE & FAULT-TOLERANT FALLBACK
OSRM Driving	HTTPS / REST	<code>router.project-osrm.org</code>	None	Road geometry & leg driving durations. Fallback: Straight-line Haversine distance with 10s timeout.
Open Charge Map	HTTPS / REST	<code>api.openchargemap.io/v3/poi</code>	API Key (<code>OCM_API_KEY</code>)	883 verified regional charging station POIs. Fallback: Persistent local disk cache with 24h TTL.
OSM Nominatim	HTTPS / REST	<code>nominatim.openstreetmap.org</code>	Custom User-Agent	Location search & autocomplete. Fallback: Pre-configured local cache of 19 transportation hubs.
OSM Overpass API	HTTPS / QL	<code>overpass-api.de/api/interpreter</code>	None	Nearby hotel/motel lodging for long journeys. Fallback: Non-blocking graceful empty list return.
Razorpay API	HTTPS / REST	<code>api.razorpay.com/v1/orders</code>	Key ID & Secret	Slot advance reservation & payment settlement. Fallback: Built-in local mock simulator for dev keys.
CartoDB Basemaps	HTTPS / Raster	<code>basemaps.cartocdn.com</code>	None	High-contrast dark map tiles for Leaflet mapping layer.
HTML5 Geolocation	Web API	<code>navigator.geolocation</code>	User Permission	Acquires current user coordinates for instant nearest-station searches.

7. Algorithms, AI & Mathematical Models

1. FIRST-PRINCIPLES BATTERY & ENERGY PHYSICS MODEL

$$\text{Usable Energy (kWh)} = \text{Capacity}_{\text{pack}} \times (\text{SoC}_{\text{start}} - \text{SoC}_{\text{reserve}}) / 100$$

$$\text{Wh/km}_{\text{adjusted}} = \text{Wh/km}_{\text{base}} \times M_{\text{occupancy}} \times M_{\text{terrain}} \times M_{\text{traffic}} \times M_{\text{weather}} + P_{\text{cargo}}$$

Occupancy: 2W solo = 1.0, 2W with pillion = 1.15 (frontal aerodynamic drag); 4W driver = 1.0, scaling +3.5% per passenger. **Cargo:** +0.04 Wh/km per kg of luggage. **Chemistry:** LFP allows 100% recharge; NMC capped at 85-90% to prevent cell degradation.

2. MULTI-CRITERIA CHARGING STATION SCORER

$$\text{Score} = w_a \cdot P_{\text{avail}} - w_d \cdot D_{\text{detour}} - w_t \cdot T_{\text{charge+queue}} + w_r \cdot R_{\text{station}} - w_c \cdot C_{\text{cost}}$$

Weights dynamically shift based on user preference: **Fastest** (favors high-power DC hubs and minimal total time), **Efficient** (minimizes Wh/km and detours), and **Reliable** (prioritizes high uptime and multi-gun CPO plazas).

3. PREDICTIVE AVAILABILITY ML MODEL (LOGISTIC REGRESSION V1)

$$P(\text{Available at ETA}) = \sigma(w \cdot x + b) = 1 / (1 + e^{-(w \cdot x + b)})$$

Infers vacancy probability at ETA by evaluating current vacancy, historical reliability score, off-peak hour curves, queue lengths, station capacity, and venue demand profile (transit, highway, mall, office, residential, campus).

8. Development, Testing & DevOps Pipeline

VoltGrid maintains rigorous continuous quality gates via automated unit testing, static typechecking, linting, and security audits.

TOOL / SYSTEM	CONFIGURATION	DETAILS & QUALITY GATES
Test Runner	Vitest 3 (<code>vitest.config.ts</code>)	Fast in-memory unit testing with 100% test pass rate across 44 automated tests.
Test Suite Coverage	<code>src/lib/__tests__/</code>	• <code>optimizer.test.ts</code> (31 tests): Stop insertion, loop avoidance, convergence. • <code>security.test.ts</code> (7 tests): Zod schemas, 128 KiB limit, rate limiter, CORS. • <code>battery-load.test.ts</code> (4 tests): Occupancy and cargo load multipliers. • <code>desired-battery.test.ts</code> (2 tests): Arrival SoC constraint guarantees.
Static Linter	ESLint 9 (<code>eslint.config.mjs</code>)	Strict Next.js 16 and React 19 recommended rules and import hygiene.
Type Checker	TypeScript Compiler (<code>tsc</code>)	<code>tsc --noEmit</code> validating zero type errors across the entire codebase.
CI/CD Workflow	GitHub Actions (<code>.github/workflows/ci.yml</code>)	Multi-stage pipeline: <code>npm ci</code> → <code>npm audit --audit-level=high</code> → <code>tsc</code> → <code>eslint</code> → <code>vitest</code> → <code>next build</code> .
Tunneling Tool	<code>cloudflared.exe</code>	Pre-packaged Cloudflare tunnel binary for local webhook verification and mobile field testing.

9. Technology Summary Matrix

ARCHITECTURAL LAYER	CORE TECHNOLOGIES & DEPENDENCIES
Frontend Layer	Next.js 16.3.2 (App Router), React 19.2.8, TypeScript 5, Tailwind CSS 4, Lucide React, CVA, clsx, tailwind-merge
Mapping & Graphics	Leaflet 1.9.4, React-Leaflet 5.0.0, React-Leaflet-Cluster 4.1.3, CartoDB Dark Tiles, Recharts 3.2.1
Server & Backend	Node.js 24 (IPv4), custom <code>server.mjs</code> , Next.js Route Handlers (16 REST API endpoints)
Security & Compliance	Zod 3/4 Runtime Schemas, In-memory Token-Bucket Limiter, Strict CORS Whitelist, CSP, HSTS, 128 KiB Body Cap
Data & Persistence	PostgreSQL + PostGIS (<code>db/schema.sql</code>), In-Memory Simulation Store, Local Disk Cache, LocalStorage
Payments & Billing	Razorpay Node SDK 2.9.8, Razorpay Checkout.js, Server HMAC-SHA256 Cryptographic Signature Verification
External APIs	OSRM Driving API, Open Charge Map POI API, OSM Nominatim Geocoding, OSM Overpass Lodging API
Engineering Algorithms	First-Principles Battery Physics, Multi-Stop Corridor Optimization, Logistic Availability ML Model
Quality & Testing	Vitest 3 (44 passing tests), ESLint 9, TypeScript 5 (<code>tsc --noEmit</code>), GitHub Actions CI/CD